ETA: Exit on Tool Arrival in Speculative Agent Loops

Abstract

Tool-using language agents in explicit thinking modes generate long chains of thought before acting, making each turn inherently serial: later tokens depend on the tool observation. Recent speculative tool dispatch methods overlap tool execution with reasoning, but offer limited benefit when reasoning dominates latency. We present **ETA**, which treats the speculative fork not as an actuator but as a sensor of the model's decision state, using it to terminate the reasoning of the main decode. It truncates reasoning only when the prediction is stable, agrees with the dispatched call, passes a free probe measuring whether a call will be emitted at all, and the tool result has arrived: a deadline that bounds per-turn regret by the remaining decode cost. ETA achieves speedups of $1.33\times$ on τ^2 airline/retail, $1.26\times$ on τ^2 telecom, and $1.13\times$ on BFCL v4 multi-turn. On τ^2, it decodes 27% fewer tokens at a nearly unchanged per-token rate without dropping action quality.

1 Introduction

Tool-using language agents increasingly rely on explicit *thinking* modes. At each turn, the model generates a chain-of-thought (CoT) and a tool call, then waits for the resulting observation [14]. This dependency serializes reasoning and tool execution, causing their latencies to accumulate over the five to fourteen turns of a typical task.

Speculative tool dispatch reduces this delay [1, 5]. When the main stream emits its first token, the controller forks a request that shares its KV-cache prefix and is constrained to predict a tool call. It executes this prediction in parallel with the remaining reasoning and uses the result only if the model eventually emits the same call. This moves tool execution off the critical path without modifying the model, a draft model, or relaxing the acceptance criterion. However, tool execution is often not the bottleneck for reasoning-heavy agents. With Qwen3 in thinking mode on τ^2-bench [15], tool execution accounts for only 4.1% of wall-clock time, while post-observation reasoning accounts for 38.9%, a nine-fold larger optimization budget (§3). Thus, the greatest opportunity for latency reduction lies in terminating post-observation reasoning, rather than merely overlapping tool execution.

We introduce ETA to recover this larger budget. We repurpose the speculative fork from a tool-launching actuator into a sensor of the model's evolving decision. Repeated along the CoT, it reveals whether the predicted call has stabilized. ETA truncates reasoning only when the predicted call is stable, matches the tool call already dispatched, passes the free-probe (§3), and the tool result is available. Finally, tool completion defines the truncation deadline. Stopping earlier only trades decode time for tool wait, while stopping at completion bounds per-turn regret by the remaining decode cost. The predicted call is never inserted into the transcript. Instead, the model derives its own action from the shortened reasoning, and the original acceptance gate verifies it against the dispatched call. Thus, the probe determines when to stop reasoning, but the model still produces the final action.

- We identify reasoning that continues after the tool result is ready as the dominant remaining cost.
- We introduce a deadline-bounded exit rule gated by prediction stability, dispatch agreement, and the free probe. Tool readiness prevents additional tool wait, no-call continuations are rolled back, and snapshot/restore supports state-mutating tools.
- ETA achieves $\mathbf{1.33}\times$ speedup on τ^2 airline/retail, $1.26\times$ on τ^2 telecom, and $1.13\times$ on BFCL v4 multi-turn [9]. On τ^2, it decodes 27% fewer tokens without a drop in quality. Action match changes by $+2.02$ pp, within the calibrated $\delta = 3.18$ pp noise floor.

Preprint.

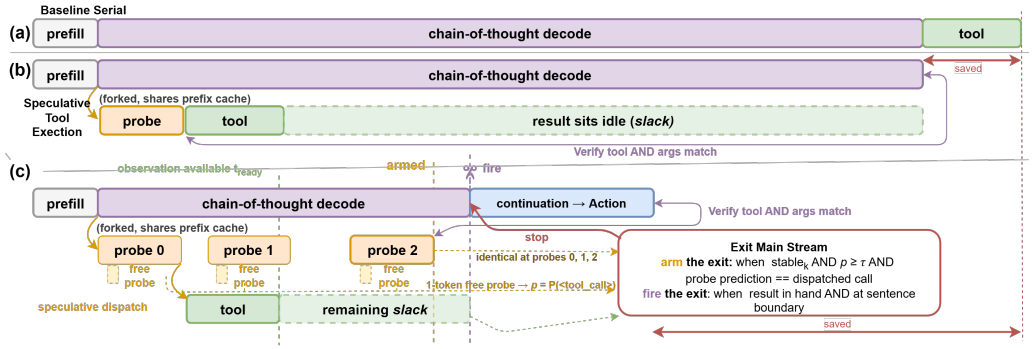


Figure 1: (a) The ReAct loop, (b) Speculative tool execution where tool execution is completely overlapped by the CoT, (c) ETA reusing the speculation to terminate reasoning of main stream early.

2 Background and Related Works

This section formalizes the ReAct loop, and the self-speculative dispatch mechanism from prior work.

The ReAct agentic loop. A Large Reasoning Model (LRM) agent operates under policy π. An episode comprises H turns, indexed by $t = 1, \ldots, H$. At turn t the policy is conditioned on a context C_t (system prompt, user request, and all previous tool observations) and greedily decodes:

$$\underbrace{r_t = (r_{t,1}, \ldots, r_{t,L_t})}_{\text{chain of thought}} \oplus \texttt{</think>} \oplus \underbrace{a_t}_{\text{action}}, \qquad a_t \in \mathcal{A} \cup \{\varnothing\}, \tag{1}$$

where an action $a_t = (u_t, v_t)$ consists of a tool name u_t with its arguments v_t, and $a_t = \varnothing$ denotes a turn that terminates in a natural-language response to the user rather than a tool-call. A tool-call takes T_t^{tool} seconds, and appends the resulting observation o_t to C_{t+1}. A serial ReAct turn therefore costs:

$$T_t^{\text{ser}} = T_t^{\text{pre}} + \underbrace{(L_t + m_t)\,\bar{d}}_{\text{decode: reasoning + action}} + \underbrace{T_t^{\text{tool}}}_{\text{tool wait}}, \tag{2}$$

where T_t^{pre} denotes prefill time, $\bar{d}$ is the mean per-token decode latency and m_t is the tool-call length in tokens. Here o_t conditions every subsequent token, so T_t^{tool} can not start until decoding completes (Figure 1a).

Self-speculative tool dispatch. Recent works SPORK, SPAgent [1, 5] remove tool wait from the critical path by predicting and dispatching tool calls before the model emits them with reasoning. Upon the main stream's first token, the controller forks a second request that shares the KV-cache prefix (prefill is a cache-hit) and is forced to produce a tool call, yielding a predicted action $\hat{a}_t$. If the fork's logprob-based confidence exceeds a threshold θ, $\hat{a}_t$ is dispatched immediately, in parallel with the main stream's remaining decode. Once the main stream eventually emits its own action a_t, the precomputed observation is used only if $\hat{a}_t \equiv a_t$ under exact tool-name and argument matching; otherwise, it is discarded and the tool re-executed serially (Figure 1b). Speculation is therefore lossless relative to serial execution because the agent never observes a mispredicted call.

Other related works. More broadly, tool-speculation methods differ in how they predict future invocations and how far ahead they speculate. Speculative Actions [16] uses a smaller draft model to predict the next action from an observation and execute it early. PASTE [11] learns recurring tool-use patterns from historical traces and replays them through a middleware interceptor. [8] instead draft tool calls within the serving engine using a smaller model and keep candidate sequences resident to avoid repeated prefill. IdleSpec [2] uses the same idle periods to generate speculative *plans*, trading additional computation for accuracy rather than latency. MORI [12] addresses a complementary systems cost by offloading the KV cache during tool waits rather than shortening the critical path. A separate line of work reduces chain-of-thought length [10]. Approaches include fixed-token budget forcing [7], reinforcement learning for length pruning [4], and dynamic exit rules. DEER generates trial answers at reasoning transitions and exits when confidence is high [13]. Dynasor/Certaindex probes the model at fixed token intervals, extracting its current answer without altering the main stream, and uses the resulting certainty signal to allocate test-time compute [3].

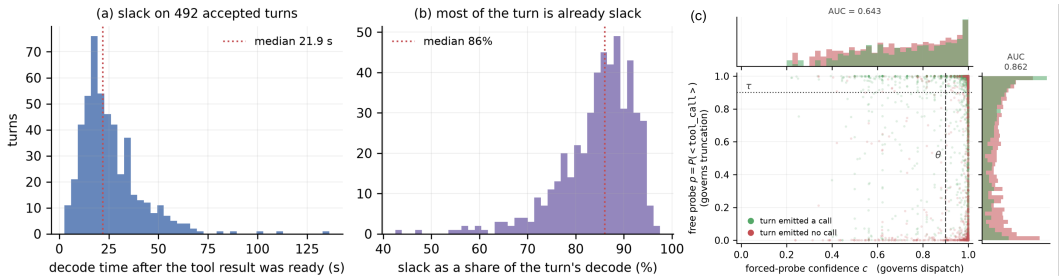


Figure 2: *(a)* Per-turn slack σ_t across accepted speculative turns, measuring decode time after the observation was available. *(b)* Slack normalized by turn-level decode time, with a median of 86%. *(c)* Forced-probe conf. c is nearly blind to tool/no-tool decision: dispatch threshold θ admits 93.4% of tool turns but also 87.4% of no-tool turns. The truncation threshold τ admits 82.3% against 28.5%.

3 ETA Methodology

When the tool execution lies on the serial critical path, its runtime fraction is $f_{\text{tool}} = \frac{1}{T}\sum_t T_t^{\text{tool}}$. Even perfect tool overlap removes only f_{tool} of runtime, limiting speedup to $1/(1-f_{\text{tool}})$. On our primary workload (τ^2-bench, Qwen3-32B in thinking mode, and a 2.0 s tool-call floor) with RTX 6000 Pro GPU, we measure $f_{\text{tool}} = 4.1\%$, implying a speedup ceiling of $\mathbf{1.043}\times$. For reasoning models that decode hundreds of tokens per turn, tool execution is therefore not the real bottleneck. This section examines what remains on the critical path after tool wait is hidden.

Exploiting the post-observation reasoning slack. Now for each overlapped turn, the slack is $\sigma_t = \left(e_t - t_t^{\text{ready}}\right)$, where e_t is when the main stream finishes decoding turn t, and t_t^{ready} is when the observation for a_t becomes available in the speculative fork. The aggregate slack fraction $f_{\text{slack}} = \frac{1}{T}\sum_t \sigma_t$ is the optimization budget targeted in this work. In the same setup, $f_{\text{slack}} = 38.9\%$, implying a speedup ceiling of $S = \frac{1}{1-f_{\text{slack}}} = \mathbf{1.64}\times$, with a $9\times$ larger optimization budget than tool-execution overlap. Per accepted turn, the median slack is 21.9 s and, as a share of that turn's decode, the median is 86% (Figure 2a-b). Truncation of main stream is permitted only once the tool observation is available: stopping earlier cannot reduce latency, since it merely swaps decode time for an equal tool-wait while risking an incorrect decision.

Free Probe. As Equation 1 shows, not every turn ends with a tool call. We must therefore distinguish assertions from tool calls before arming the exit mechanism. Prior work uses confidence over the probe's tool-name span to decide whether to speculatively dispatch, but this confidence cannot serve as an exit signal. Because the forced-probe prompt ends *inside* a tool call (A.3 left), its continuation is restricted to tool names. Thus, $c^{(j)} \approx P_\pi(u = \hat{u}^{(j)} \mid \text{a call is emitted})$ measures confidence in the predicted tool *conditional* on a call, but provides no evidence that a call should occur. When the correct outcome is an assertion, the probe cannot be correct, yet may remain highly confident in the tool name it is forced to generate. High probe confidence is insufficient for safely terminating the reasoning. For solution, we create an additional probe by removing the final two prompt lines (A.3 right). This places the model at a genuine decision point: its next token is either `<tool_call>` or the first token of the assertion. We request *exactly one token* with `logprobs=20` and measure the probability mass assigned to `<tool_call>`.

$$p^{(j)} \;=\; \exp \ell\big(\texttt{<tool_call>}\big) \cdot \mathbb{1}\big[\texttt{<tool_call>} \in \text{top-20}\big]. \tag{3}$$

This probe costs a single decode step for prefix cache hit. Although both probes read the same model state, c identifies *which tool* from inside the call (to dispatch the tool), whereas p determines *whether to call a tool* at the `</think>` boundary. In A.2, Turn 4 of a worked example demonstrates the utility of devising the free probe, which prevents an incorrect action due to early exit. Figure 2(c) confirms the same effect at scale. Across all 7133 probes of the τ^2-suite, c poorly distinguishes turns that end in an action from those that do not (AUC 0.643), whereas p separates them effectively (AUC 0.862).

The deadline-bounded exit mechanism. In (Figure 1c) probes are forked at CoT positions $n_0 = 0 < n_1 < \cdots < n_J$, spaced $\Delta = 50$ tokens apart and snapped to sentence boundary with max $J = 15$ probes per turn (A.4). Let $\hat{a}^{(j)} = (\hat{u}^{(j)}, \hat{v}^{(j)})$ denote the action predicted by probe j. We write $\hat{a}^{(j)} \equiv a^{(k)}$ when their tool names and arguments match exactly, following the acceptance-gate criterion, and use $\breve{a}_t$ for the dispatched call (whenever the dispatch threshold was satisfied). We

define stability and the arming condition as follows:

$$\text{stable}_k(j) = \mathbb{1}\Big[\hat{a}^{(j)} \equiv \hat{a}^{(j-1)} \equiv \cdots \equiv \hat{a}^{(j-k+1)}\Big], \tag{4}$$

$$A_j = \text{stable}_k(j) \ \wedge \ \big(p^{(j)} \geq \tau\big) \ \wedge \ \big(\hat{a}^{(j)} \equiv \check{a}_t\big). \tag{5}$$

Arming is necessary but insufficient. Let $\text{ready}_t(n)$ indicate that speculative execution has completed by the time token n is produced, and let $\text{bnd}(\cdot)$ indicate a sentence boundary. We fire the cut point at:

$$n^\dagger = \min\Big\{ n \ : \ \text{ready}_t(n) \ \wedge \ \text{bnd}(r_{t,1:n}) \ \wedge \ \big(\exists\, j \leq J : \ A_j \wedge n \geq n_j\big)\Big\}. \tag{6}$$

At $n^\dagger$, we cooperatively stop the main stream and conclude the turn using:

$$a_t \sim \pi\big(\cdot \,\big|\, C_t,\ r_{t,1:n^\dagger} \oplus \texttt{</think>}\big). \tag{7}$$

We append only `</think>` and never inject $\check{a}_t$ into the main stream. Thus, truncation cannot force the model to emit an unintended action. Figure 3 illustrates the mechanism, while Alg. 1 specifies the complete procedure. A.5 provides additional rationale for the exit mechanism.

4 Evaluation and Results

Our setup uses Qwen3-32B in thinking mode, served by vLLM [6], with a 2.0 s tool-call latency floor. We compare five methods that differ only in the mechanism under test: **serial** (no speculation), **SPORK (minimal)** [1], **SPORK** (confidence-gated retry probing; strongest baseline), **ours (off)** (full instrumentation: pays the extra cost but takes no benefit), and **ours (on)** (truncation enabled).

Table 1 compares the five methods across all 155 τ^2-bench airline and retail tasks, comprising 692 gold actions. Table 2 reports every pairwise speed and quality contrast. The treatment effect on action match is smaller than the largest calibration contrast expected to be zero (+2.02 vs. $\delta = 3.18$ pp), while every lower bound remains above the non-inferiority margin $-\delta$ (A.6). The speedup is driven by decoding fewer tokens rather than better tool overlap: wall-clock time per token changes by only -0.2%, while SPORK reduces per-token time but emits more tokens overall, negating its gain (Table 1). The improvement is not carried by outliers (median per-task speedup 1.28×; A.7).

Table 1: Five methods, 155 τ^2 tasks, 692 golden actions. Wall-time in ks, decode tokens in k

method	wall	P50	P95	match(%)	turns	tok	ms/tok
serial	31.5	163	510	48.7	785	599	52.60
SPORK	32.5	174	478	51.9	824	635	51.13
SPORK (retry)	31.7	170	457	50.7	839	620	51.05
ours (off)	33.4	171	495	48.3	797	644	51.86
ours (on)	**23.7**	**125**	**412**	**52.7**	871	**453**	52.47

Table 2: Every contrast is *baseline* → ours (on) on the same 155 tasks ; calibrators in A.6.

	wall-clock			quality	
baseline	saved	95% CI	speedup	Δ pp	LCB
SPORK (retry)	**+25.0%**	[+16.1,+32.8]	**1.334×**	**+2.02**	−2.22
ours (off)	+29.0%	[+21.6,+35.5]	1.408×	+4.48	+0.83
serial	**+24.6%**	[+15.4,+32.6]	**1.327×**	**+4.05**	+0.78
SPORK	+26.9%	[+18.4,+34.3]	1.369×	+0.87	−3.11

Results on other tasks. The operating point ($k = 3$, $\tau = 0.9$, dispatch-match) was selected offline and applied to Table 3's workloads without retuning (A.8). The method generalizes to τ^2 telecom-solo (average 13.8 turns/task), and BFCL, which is stateful where wrong-state failures fall from 47 to 37 alongside the speedup (1.258× and 1.134× respectively, against baseline).

Table 3: Three workloads, one frozen rule; paired task-level bootstrap.

		speedup [95% CI]		
workload	n	vs. baseline	vs. serial	quality
τ^2 airline+retail	155	**1.334**× [1.19, 1.49]	1.327× [1.18, 1.48]	**+2.02** pp, non-inf.
τ^2 telecom-solo	40	1.258× [1.143, 1.391]	**1.326**× [1.194, 1.474]	**+8.70** pp, non-inf.
BFCL v4 multi-turn	200	**1.134**× [1.071, 1.215]	1.133× [1.022, 1.258]	**+2.59** pp, non-inf.

Conclusion. After speculative dispatch hides tool latency, post-observation reasoning becomes the dominant remaining cost. ETA uses the speculative fork to detect action commitment and safely truncate this reasoning under a bounded-latency deadline. ETA preserves non-inferior action quality while achieving 1.33×, 1.26×, and 1.13× speedups across three workloads against state-of-the-art baseline. Limitations and future works are in A.9.

References

[1] Huajun Bai, Weiwei Lv, Huichuan Zheng, Youyou Lu, and Jiwu Shu. Spork: Self-speculative forking to accelerate agentic llm inference. *arXiv preprint arXiv:2607.03333*, 2026.

[2] Daewon Choi et al. IdleSpec: Exploiting idle time via speculative planning for LLM agents. *arXiv preprint arXiv:2605.22154*, 2026.

[3] Yichao Fu, Junda Chen, Siqi Zhu, Zheyu Fu, Zhongdongming Dai, Aurick Qiao, and Hao Zhang. Efficiently serving LLM reasoning programs with Certaindex. *arXiv preprint arXiv:2412.20993*, 2024.

[4] Bairu Hou, Yang Zhang, Jiabao Ji, Yujian Liu, Kaizhi Qian, Jacob Andreas, and Shiyu Chang. ThinkPrune: Pruning long chain-of-thought of LLMs via reinforcement learning. *arXiv preprint arXiv:2504.01296*, 2025.

[5] Zixiao Huang, Wen Zeng, Tianyu Fu, Tengxuan Liu, Yizhou Sun, Ke Hong, Xinhao Yang, Chengchun Liu, Yan Li, Quanlu Zhang, et al. Reducing latency of llm search agent via speculation-based algorithm-system co-design. *arXiv preprint arXiv:2511.20048*, 2025.

[6] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pages 611–626, 2023.

[7] Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. s1: Simple test-time scaling. *arXiv preprint arXiv:2501.19393*, 2025.

[8] Daniel Nichols, Prajwal Singhania, Charles Jekel, Abhinav Bhatele, and Harshitha Menon. Optimizing agentic language model inference via speculative tool calls. *arXiv preprint arXiv:2512.15834*, 2025.

[9] Shishir G. Patil, Huanzhi Mao, Charlie Cheng-Jie Ji, Fanjia Yan, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models. In *Forty-second International Conference on Machine Learning*, 2025.

[10] Yang Sui, Yu-Neng Chuang, Guanchu Wang, Jiamu Zhang, Tianyi Zhang, Jiayi Yuan, Hongyi Liu, Andrew Wen, Shaochen Zhong, Hanjie Chen, and Xia Hu. Stop overthinking: A survey on efficient reasoning for large language models. *Transactions on Machine Learning Research (TMLR)*, 2025. arXiv:2503.16419.

[11] Yang Sui, Hao Zhao, Rui Ma, Zhiwei He, Hao Wang, Jian Li, and Yi Yang. Act while thinking: Accelerating LLM agents via pattern-aware speculative tool execution. *arXiv preprint arXiv:2603.18897*, 2026.

[12] Tian Xia, Hanchen Li, Zhifei Li, Xiaokun Chen, Hao Kang, Yifan Qiao, Yi Xu, and Ion Stoica. Idleness is relative: Exploiting tool-call idle windows for offloading in agentic systems with mori. *arXiv preprint arXiv:2606.00866*, 2026.

[13] Chenxu Yang, Qingyi Si, Yongjie Duan, Zheliang Zhu, Chenyu Zhu, Qiaowei Li, Mingyu Chen, Zheng Lin, and Weiping Wang. Dynamic early exit in reasoning models. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2504.15895.

[14] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*, 2023. URL `https://openreview.net/forum?id=WE_vluYUL-X`.

[15] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains, 2024. URL `https://arxiv.org/abs/2406.12045`.

[16] Naimeng Ye, Arnav Ahuja, Georgios Liargkovas, Yunan Lu, Kostis Kaffes, and Tianyi Peng. Speculative actions: A lossless framework for faster agentic systems. *arXiv preprint arXiv:2510.04371*, 2025.

A Appendix

A.1 Glossary

H	turns per episode	c, θ	span confidence, dispatch threshold
T	wall-clock	p, τ	free-probe signal, truncation threshold
$\bar{d}$	mean per-token decode latency	α	speculation acceptance rate
T_t^{tool}	tool execution time at turn t	σ_t	per-turn slack (§3)
f_{tool}	tool time as a fraction of T	f_{slack}	slack as a fraction of T

A.2 Worked Example

This four-turn task illustrates successful truncation and two cases in which a probe-side guard prevents an incorrect exit. The user asks to exchange a bookshelf for a camera. The agent must authenticate the user, retrieve the order, determine that it has not yet been delivered, and decline the exchange. The final column reports either the token at which generation was truncated or the guard that prevented truncation.

turn	main stream emits	forced probe predicts	p over probes	outcome
1	`find_user_id`	*same*, $\times 3$	.999 1.00 1.00	**cut** at token 173
2	`get_user_details`	*same*, $\times 3$	.979 1.00 1.00	**cut** at token 158
3	`get_order(#. . . 037)`	`#. . . 222`, then `#. . . 037`	.986 . . . 1.00	dispatch mismatch
4	*message, no call*	`transfer_to_human`, $\times 5$	.000 $\times$ 5	$p < \tau$

Turns 1 and 2 show the intended behavior. In both cases, the prediction is exact and quickly becomes stable. The exit therefore arms at probe 2 and fires at the next probe boundary. Truncation reduces chain-of-thought length from 386 to 176 tokens on turn 1 and from 493 to 160 tokens on turn 2. Together, the two turns take 20.8 s, compared with 46.9 s without truncation, including continuation costs of 2.1 s and 1.5 s.

Turns 3 and 4 show why the remaining guards are necessary. On turn 3, probe 0 predicts the nonexistent order ID `#W5362222`, and a request (tool-call) for that order is dispatched. Probes 1-3 then converge on the correct ID, `#W5362037`, with $p = 1.000$. Although the stability and free-probe conditions are satisfied, the dispatch-match guard blocks the exit because the in-flight request concerns a different, fictitious order. Truncating at this point would therefore provide no useful result.

On turn 4, the forced probe repeatedly predicts `transfer_to_human_agents`, producing the same 54-word summary five times. From probe 2 onward, this prediction is both stable and identical to the dispatched call, so stability and dispatch matching alone would permit an exit. The main stream, however, ultimately sends a message to the user without making any tool call. The free probe correctly blocks truncation: it assigns $p = 0.000$ at every probe, with `'YOU'` as the top token at probability 0.999. This example makes the gap between the forced probe's conditional prediction and the main stream's marginal decision concrete.

A.3 Free Probe Mechanics

forced probe: *which* tool?

```
. . . <|im_start|>assistant
<think>
Okay, let's see. The user is James. . .
</think>
<tool_call>
{"name": "                                ← ends here
```

free probe: *whether* a tool?

```
. . . <|im_start|>assistant
<think>
Okay, let's see. The user is James. . .
</think>
                                          ← ends here
```

A.4 Probe Cadence

Probe 0 is issued when the main stream emits its first token. Each subsequent probe is issued after the stream decodes another $\Delta = 50$ CoT tokens and reaches a sentence boundary, identified by a trailing

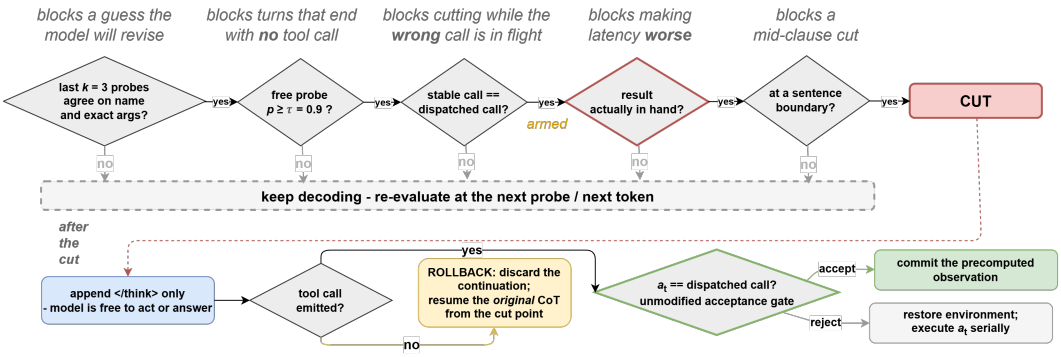


Figure 3: Overview of the decision mechanism for terminating main-stream reasoning and the failure mode blocked by each guard. The probe prediction is never adopted. After truncation, the model derives its own action, which must pass the unmodified acceptance gate of main stream. A turn that declines to make a tool-call is rolled back onto its original reasoning.

\n, ". ", or ".\n". If no boundary occurs within an additional $\Delta_g = 30$ tokens, the probe is issued immediately. We issue at most $J = 15$ probes per turn. Thus, the cadence is primarily token-driven; sentence boundaries determine where near the target interval a probe is placed. Each probe requires two prefix-cache-hit requests: a forced probe of at most 160 tokens and a one-token free probe.

Previous works stop probing after its first confident dispatch. We therefore continue probing after dispatch. Exit mechanism disabled (ours (off) in §4) incurs the full cost of these additional probes while disabling truncation, which isolate the benefit of the exit rule.

A.5 Design of the Exit Mechanism

Figure 3 summarizes the exit mechanism. Three design choices in Eqs. (6) and (7) are central to its behavior.

First, readiness is observed rather than estimated. The controller evaluates ready_t by polling whether the speculative tool execution has completed. The deadline is therefore exact and requires no prior model of tool latency. In the example in Figure 1(a), the tool result is available well before the exit rule permits truncation. At this latency, the mechanism is limited by prediction stability rather than tool readiness; the deadline begins to bind only as T^{tool} increases.

Second, the continuation remains unconstrained. After truncation, we append only </think> and allow the model to generate its own action; we never inject the dispatched call $\check{a}_t$. The mechanism therefore cannot manufacture an action on the model's behalf, and action-level quality continues to measure the model's behavior.

Third, the exit guards do not replace the acceptance gate. Equation (5) compares the probe prediction with the dispatched call. The acceptance gate separately compares the model's final call with the dispatched call. Thus, truncation cannot force acceptance: if the continuation emits a different action, the speculative result is discarded and the emitted action is executed normally.

No-tool rollback. A continuation may emit no tool call even after satisfying all exit guards, indicating that the free probe incorrectly predicted a tool-emitting turn. In this case, truncation may have interrupted a terminal answer. We found no reliable way to identify such cases at exit time: across 104 truncations, harmful exits were not separable by token count, p, confidence, or probe index, and candidate thresholds rejected substantially more beneficial exits than harmful ones.

We therefore detect this outcome directly and roll it back without a threshold. The controller discards the continuation and resumes the original reasoning from $n^{\dagger}$ without appending </think>. Because this is a warm-cache continuation, it only regenerates tokens that the untruncated stream would otherwise have produced. The mechanism can consequently incur additional latency, but a no-tool continuation is not committed as the turn's final output.

Operating point. We selected $k = 3$, $\tau = 0.9$, and exact dispatch matching offline, before the full task scale evaluations. We replayed recorded probe streams under candidate rules and measured

Algorithm 1 One agent turn under ETA

Require: context C_t; thresholds θ (dispatch), τ (truncation); stability k; cadence Δ, Δ_g; probe budget J

1: launch main stream $\pi(\cdot \mid C_t)$ with a cooperative stop hook
2: $\breve{a} \leftarrow \varnothing$; $\mathcal{P} \leftarrow [\,]$ ▷ dispatched call; probe history
3: **wait** for first streamed token
4: **for** $j = 0, \ldots, J-1$ **do**
5: **if** $j > 0$ **then wait** until Δ further CoT tokens **and** sentence boundary (or $\Delta + \Delta_g$ tokens)
6: **end if**
7: $r_{1:n_j} \leftarrow$ snapshot of main's CoT
8: $(\hat{a}^{(j)}, c^{(j)}) \leftarrow \textsc{ForcedProbe}(C_t, r_{1:n_j})$ ▷ cache hit
9: $p^{(j)} \leftarrow \textsc{FreeProbe}(C_t, r_{1:n_j})$ ▷ 1 token, logprobs=20, Eq. (3)
10: append $(\hat{a}^{(j)}, c^{(j)}, p^{(j)})$ to $\mathcal{P}$
11: **if** $\breve{a} = \varnothing$ **and** $c^{(j)} \geq \theta$ **then**
12: $\breve{a} \leftarrow \hat{a}^{(j)}$; $\textsc{Snapshot}$(env); $F \leftarrow \textsc{DispatchAsync}(\breve{a})$
13: **end if**
14: **if** A_j holds (Eq. (5)) **then**
15: armed $\leftarrow$ True; **break**
16: **end if**
17: **end for**
18: **on each streamed token** n**: if** armed $\wedge$ F.DONE() $\wedge$ bnd$(r_{1:n})$ **then** stop main at $n^\dagger \leftarrow n$
19: **if** main was truncated **then**
20: $a_t \leftarrow \textsc{Complete}(C_t, r_{1:n^\dagger} \oplus$ `</think>`$)$ ▷ free continuation
21: **if** $a_t = \varnothing$ **then**
22: $a_t \leftarrow \textsc{Complete}(C_t, r_{1:n^\dagger})$ ▷ rollback: resume original CoT
23: **end if**
24: **else**
25: $a_t \leftarrow$ main's own emitted action
26: **end if**
27: **if** $a_t \equiv \breve{a}$ **then** commit F's observation **else** $\textsc{Restore}$(env); execute a_t serially

two quantities: *precision*, the fraction of proposed truncations whose predicted call matched the call produced by the untruncated turn; and *harvest*, the wall-clock time those truncations would save. On τ^2-bench, the selected operating point achieved 0.917 precision while targeting 34.4% of wall-clock time. We then froze the rule and applied it without retuning to HotpotQA and BFCL v4 Multi-Turn, where precision increased to 0.938 and 0.976, respectively.

Speculation with state-mutating tools. Prior speculative-dispatch methods are limited to read-only tools because an incorrect speculative call could alter the state on which the run is evaluated. Two of our workloads instead use mutable backends, including a file system and a trading account. Before dispatch, we snapshot the backend and execute the speculative call on an isolated copy. The resulting state is committed only if the acceptance gate matches the speculative call to the model's final call; otherwise, the original state is restored (Alg. 1, lines 11 and 20). Speculation therefore affects only latency until acceptance, extending the mechanism to state-mutating tools without relaxing the acceptance criterion.

A.6 Non-inferiority analysis

We evaluate action match using a paired, task-level percentile bootstrap with $20\,000$ resamples. Resampling tasks preserves within-task dependence; treating turns as independent would produce intervals that are too narrow. For each resample, we compute

$$\frac{\sum_i m_i^B - \sum_i m_i^A}{\sum_i g_i},$$

where m_i^A and m_i^B are matched actions and g_i is the number of golden actions for task i. Resampling both numerator and denominator avoids giving equal weight to tasks with different numbers of actions.

We use action match rather than task success because τ^2 assigns the same task-level score to trajectories matching 9 of 10 actions and 0 of 10. Across the five arms, only 26–30 of 155 tasks succeed, whereas action match provides 692 scored events and approximately $4.5\times$ greater resolution.

Calibrating the margin. We estimate the apparatus noise from comparisons that should not change model decisions. Before examining the treatment effect, we set the non-inferiority margin to the largest absolute difference among these calibrators:

calibrator	Δ actions	Δ pp	expected equivalence
serial $\rightarrow$ SPORK	$+22$	$\mathbf{+3.18}$	lossless speculation
serial $\rightarrow$ SPORK(retry)	$+14$	$+2.02$	unchanged acceptance gate
SPORK $\rightarrow$ SPORK(retry)	-8	-1.16	unchanged acceptance gate
SPORK(retry) $\rightarrow$ ours (off)	-17	-2.46	instrumentation only

These differences arise from serving nondeterminism, including changes in batching, KV-cache state, and floating-point reduction order. We therefore set $\delta = 3.18$ pp. Effects smaller than this margin cannot be resolved reliably by the present experiment.

The same contrasts calibrate the latency axis, and none of them moves it: wall-clock changes by -3.1% [-14.7, $+6.9$], $+2.5\%$ [-5.0, $+9.7$] and -5.5% [-16.1, $+4.1$] for serial $\rightarrow$ SPORK, SPORK $\rightarrow$ SPORK(retry) and SPORK(retry) $\rightarrow$ ours (off) respectively (equivalently $0.970\times$, $1.026\times$, $0.948\times$). Every interval contains zero, so the apparatus does not manufacture speedups on comparisons that should show none, whereas every treatment interval in Table 2 excludes it.

Non-inferiority results. A comparison is non-inferior when its one-sided 95% lower confidence bound (LCB) exceeds $-\delta$.

comparison	Δ pp	95% LCB	verdict
SPORK(retry) $\rightarrow$ ours (on)	$+2.02$	-2.22	**non-inferior**
serial $\rightarrow$ ours (on)	$+4.05$	$+0.78$	non-inferior; LCB > 0
SPORK $\rightarrow$ ours (on)	$+0.87$	-3.11	non-inferior
ours (off) $\rightarrow$ ours (on)	$+4.48$	$+0.83$	non-inferior; LCB > 0

All treatment comparisons satisfy non-inferiority at $\delta = 3.18$ pp. Against SPORK(retry), our method is $1.33\times$ faster with an action-match difference of $+2.02$ pp and an LCB of -2.22 pp. We therefore conclude that any quality loss is smaller than the experiment can resolve.

A.7 Per-task distributions and mechanism behavior

Per-task speedup. Against SPORK(retry), our baseline, the median task-level speedup is $\mathbf{1.28}\times$ (P05 $0.43\times$, P25 $1.01\times$, P75 $1.91\times$, P95 $4.40\times$). Our method is faster on $118/155$ tasks (76%), including 34 tasks with at least a $2\times$ speedup. It is slower on 37 tasks (24%), including 9 by at least $2\times$. The worst case is `airline_7` at $0.21\times$; the largest absolute regression is `retail_113` at $+305$ s.

arm	P10	P25	P50	P75	P90	P95	P99
serial	80	109	163	233	421	510	667
SPORK	91	121	174	250	412	478	542
SPORK(retry)	90	115	170	237	393	457	540
ours (on)	**64**	**90**	**125**	**197**	**267**	**412**	**484**

The improvement extends across the latency distribution. As Figure 4(a–b) shows, relative to SPORK(retry), P90 falls from 393 to 267 s (-32%), and P95 falls from 457 to 412 s. This is an aggregate result, not a per-task guarantee: the deadline applies per turn, and truncation can change the trajectory or add turns.

Where truncation occurs. Figure 4(c) shows that truncated turns are substantially shorter than the SPORK(retry) baseline:

group	n	mean	P50	P90
SPORK(retry), all turns	839	708	545	1 150
ours, not truncated	282	854	618	1 340
ours, truncated	589	307	254	490

Across all turns, our method emits 484 chain-of-thought tokens per turn, compared with 708 for SPORK(retry) (-32%). Despite using 3.8% more turns, it reduces total chain-of-thought tokens by 29% and total decoded tokens by 27%. Non-truncated turns are longer than the baseline average, suggesting that the exit selectively fires when the model settles on an action early.

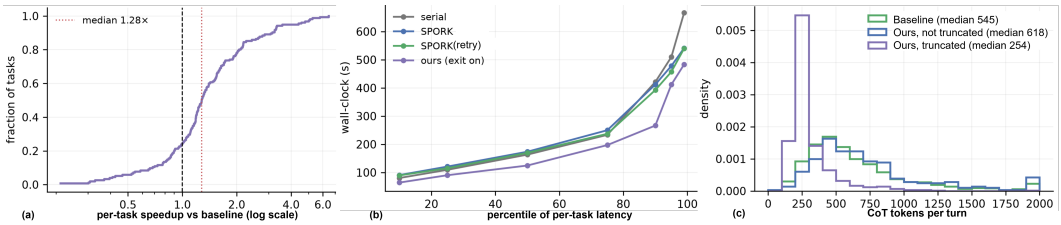


Figure 4: **Task-level latency and truncation behavior.** *(a)* Distribution of per-task speedup over SPORK(retry), with a median of $1.28\times$. *(b)* Task-latency percentiles by arm; our method reduces latency throughout the distribution. *(c)* Chain-of-thought length for SPORK(retry) and for truncated and non-truncated turns. Truncation targets turns on which the action stabilizes early.

Dispatch outcomes. The following outcomes partition all dispatched speculations:

outcome	SPORK	SPORK(retry)	ours (on)
accepted (α): name and arguments match	492 (61.3%)	531 (65.2%)	**567 (68.6%)**
rejected: main stream emits a different call	173 (21.6%)	147 (18.1%)	137 (16.6%)
rejected: main stream emits no call	137 (17.1%)	136 (16.7%)	123 (14.9%)
total	802	814	827

The exit guards directly address mismatched calls. No-call rejections remain a separate limitation: the speculative path dispatches a tool, but the main stream ultimately responds to the user without calling one.

A.8 Offline selection of the exit rule

We selected the exit rule by replaying recorded probe streams under candidate settings. Each setting was evaluated on **precision**, the fraction of proposed truncations whose predicted call matched the untruncated call, and **harvest**, the fraction of wall-clock time it could remove. The selected rule ($k = 3$, $\tau = 0.9$, with exact dispatch matching) achieved 0.917 precision and targeted 34.4% of wall-clock time on τ^2-bench. We fixed this rule before the full evaluations and performed no online tuning.

Transfer without retuning. The same rule achieved precision of 0.938 on HotpotQA and 0.976 on BFCL. Precision increased as arguments became easier to predict: τ^2 primarily uses identifiers already in context, HotpotQA composes free-text queries, and BFCL uses enums, numeric values, and identifiers from prior observations.

A.9 Limitations and Future Work

While ETA delivers substantial token and latency savings, we identify four core systemic constraints:

- **Inference Slack Availability:** ETA targets multi-turn loops with high tool usage. On conversational workloads with minimal tool calls, the available slack approaches zero. Premature truncation in these environments can occasionally induce an extra trailing turn, degrading performance.
- **Upstream Dispatch Inefficiencies:** Our framework optimizes inference only after an action is actively launched. It cannot resolve structural errors where the initial speculative fork triggers an unnecessary tool call that the main branch ultimately rejects (accounting for 14.9% of dispatches on τ^2-bench).
- **Complex Argument Synthesis:** The generation of complex, free-form, or nested tool arguments delays sequence stabilization (A_j). This extended token distance narrows the available optimization window between prediction consensus and tool readiness (t_{ready}).
- **vLLM Scheduler Interference:** Executing intermediate auxiliary token probes alters the runtime batch composition inside vLLM. While prefix-cache hits keep processing overhead bounded, high concurrent traffic introduces minor scheduling friction that requires deep algorithm-system co-design.

NeurIPS Paper Checklist

1. **Claims**
   - **Question:** Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?
   - **Answer:** [Yes]
   - **Justification:** Our abstract and introduction outline the core contribution of identifying inference slack and evaluating the ETA mechanism, matching the experimental results in Section 4.
2. **Limitations**
   - **Question:** Does the paper discuss the limitations of the work performed by the authors?
   - **Answer:** [Yes]
   - **Justification:** We explicitly provide a dedicated, comprehensive evaluation of algorithmic and infrastructure constraints in Appendix A.9.
3. **Theory assumptions and proofs**
   - **Question:** For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?
   - **Answer:** [Yes]
   - **Justification:** Section 3 provides the exact mathematical problem formulation (Eq. 1), serial latency equations (Eq. 2), and the cost-model calculations used to derive our theoretical execution boundaries.
4. **Experimental result reproducibility**
   - **Question:** Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?
   - **Answer:** [Yes]
   - **Justification:** Section 4 provides full configuration parameters, including the underlying LLM variant (Qwen3-32B), the inference framework (vLLM), execution limits (2.0s latency floor), and target baseline definitions.
5. **Open access to data and code**
   - **Question:** Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?
   - **Answer:** [No]
   - **Justification:** As this is a single-blind submission for an extended abstract workshop, the complete source code repository and orchestration harness will be fully released publicly upon formal final publication.
6. **Experimental setting/details**
   - **Question:** Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?
   - **Answer:** [Yes]
   - **Justification:** Section A.4 and A.5 list the exact operational hyperparameters chosen for the exit rule ($k = 3, \tau = 0.9$, token cadence $\Delta = 50$), and explains how they were chosen offline via recorded probe stream replay.
7. **Experiment statistical significance**
   - **Question:** Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?
   - **Answer:** [Yes]
   - **Justification:** In Section 4 and Table 2, we report explicit 95% confidence intervals (CIs) and one-sided Lower Confidence Bounds (LCBs) derived via paired task-level bootstrap resampling.

8. **Experiments compute resources**
    - **Question:** For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?
    - **Answer:** [Yes]
    - **Justification:** Section 3 specifies our concrete infrastructure setup, identifying the target workstation cluster as a 4-GPU system utilizing NVIDIA RTX 6000 Pro GPUs.
9. **Code of ethics**
    - **Question:** Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics?
    - **Answer:** [Yes]
    - **Justification:** We have reviewed the guidelines; our work focuses entirely on local infrastructure computational efficiency optimizations and uses open-source datasets.
10. **Broader impacts**
    - **Question:** Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?
    - **Answer:** [Yes]
    - **Justification:** The primary positive impact is reducing the massive carbon footprint and economic costs of serving long-reasoning agents; we do not foresee any direct unique negative societal risks arising from this infrastructure optimization.
11. **Safeguards**
    - **Question:** Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?
    - **Answer:** [N/A]
    - **Justification:** This work does not release or train new generative models, assets, or scraped datasets.
12. **Licenses for existing assets**
    - **Question:** Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?
    - **Answer:** [Yes]
    - **Justification:** We cite and credit the original creators of the underlying benchmarks (τ^2-bench) and baseline speculation concepts (SPORK) within our references.
13. **New assets**
    - **Question:** Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?
    - **Answer:** [N/A]
    - **Justification:** No new models or data distributions are introduced; our architecture optimizes existing agent inference logic.
14. **Crowdsourcing and research with human subjects**
    - **Question:** For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?
    - **Answer:** [N/A]
    - **Justification:** This research does not involve any human subjects, annotators, or crowdsourced labor.
15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**
    - **Question:** Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

- **Answer:** [N/A]
- **Justification:** No human participant experiments were conducted.

16. **Declaration of LLM usage**
    - **Question:** Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research?
    - **Answer:** [Yes]
    - **Justification:** Our method directly interfaces with and modifies the inner decoding operations of LLMs via token probe probing, which we detail throughout Section 3.